

Phase 02 - Week 06 - Day 02 - 06Sep2026

# Resume RAG Enhancement — Assignment Report

**\*\*Project:\*\*** RecruitBot — Enhanced Resume RAG Application

---

## ## 1. Assignment

Enhance the existing Resume RAG application by implementing:

- Re-ranking of retrieved resumes
- Deduplication of repeated results
- Summarization of the final results
- Complete end-to-end pipeline integration
- Display all relevant results clearly on the UI

And customize with organization branding: name and logo, brand theme and colours, professional UI layout, responsive design, and loading / success / validation / error messages. Then test the complete end-to-end application.

---

## ## 2. Solution Overview

RecruitBot is a recruiter-focused web application. A recruiter describes a role in plain language and receives **\*\*ranked, deduplicated, AI-summarized\*\*** candidate matches from a resume database.

**\*\*Two applications:\*\***

| App                            | Tech                                                                         | Port | Role                               |
|--------------------------------|------------------------------------------------------------------------------|------|------------------------------------|
| Backend (`resume-rag-backend`) | Node + Express + TypeScript, MongoDB Atlas, Mistral (embeddings), Groq (LLM) | 3000 | Retrieval pipeline + ingestion API |
| Frontend (`recruitbot-web`)    | Vite + React 18 + TypeScript + TailwindCSS + Zustand                         | 5173 | Search chat UI + upload UI         |

**\*\*Enhanced retrieval pipeline\*\*** — one call to `POST /v1/search` runs:

...

Query

- Embed (Mistral)
- BM25 search + Vector search (in parallel)
- Merge + Deduplicate (by resume id)

- Re-rank with LLM (Groq) — relevance score + reason
- Summarize each result (Groq) — fit summary
- Ranked, deduplicated, summarized results
- ...

The database currently holds **168 resumes**, all embedded and searchable.

---

## ## 3. Features Implemented

### ### 3.1 Re-ranking of retrieved resumes

After retrieval, an LLM re-orders candidates by how well they fit the query, assigning each a **relevance score** and a short **"why"** reason. Results are shown in ranked order (#1, #2, ...) with a coloured score pill and the reason on each card.

### ### 3.2 Deduplication of repeated results

Keyword (BM25) and semantic (Vector) searches often return the same candidate. These are **merged into single results** keyed by resume id. The UI shows this explicitly:

- A results-header line: **"Deduplicated — N candidates matched by both keyword + semantic search (merged)."**
- A per-card **"Keyword + Semantic"** badge.

### ### 3.3 Summarization of final results

Each top candidate gets an AI **"Fit Summary"** tailored to the query (shown truncated on the card, in full in the profile modal).

### ### 3.4 Complete end-to-end pipeline integration

A single search triggers the whole chain (embed → retrieve → dedupe → re-rank → summarize) in one request, with per-stage timings and graceful degradation if the LLM is briefly unavailable.

### ### 3.5 Display all relevant results clearly on the UI

- Results header: "Found N candidates · Mode · time"
- Ranked candidate cards: rank badge, score pill, role, experience, dedup badge, skills, "why" reason, fit summary
- A candidate **profile modal** (opens on card click; closes via ✕, overlay click, or Esc)
- Graceful **empty state** and honest low-relevance results for off-topic queries

---

## ## 4. Organization Branding & UX Customization

| Aspect        | Choice                                                                                             |
|---------------|----------------------------------------------------------------------------------------------------|
| Name          | <b>RecruitBot</b>                                                                                  |
| Logo          | <b>"RB"</b> monogram in a blue→cyan gradient (also the favicon)                                    |
| Theme         | <b>Professional Blue</b> , dark — primary <code>#2563eb</code> , accent <code>#06b6d4</code>       |
| Backgrounds   | Cool blue-gray — <code>#0b1120</code> / <code>#111a2e</code> / <code>#172136</code>                |
| Score colours | Vector <code>#3b82f6</code> , BM25 <code>#22d3ee</code> , Hybrid <code>#34d399</code>              |
| Layout        | Sidebar (brand, search modes, AI enhancement toggles, results limit) + chat main + candidate modal |
| Responsive    | Mobile (<768px) collapses the sidebar into a slide-out drawer with a hamburger toggle              |
| Messages      | Loading, success, validation, and error states across search + upload                              |

**Search modes:** Vector (semantic), BM25 (keyword), Hybrid (both, with adjustable weights). **AI Enhancements:** Re-ranking and Summarize toggles.

---

## ## 5. Messaging (loading / success / validation / error)

| Type        | Where  | Example                                                                              |
|-------------|--------|--------------------------------------------------------------------------------------|
| Loading     | Search | "Searching & ranking candidates..." with animated dots                               |
| Loading     | Upload | Progress bar → pipeline stepper (upload → extract → parse → embed → store)           |
| Success     | Upload | Checklist screen + parsed summary                                                    |
| Success     | Search | "Found N candidates · Mode · time" header                                            |
| Validation  | Upload | "Only PDF allowed", "Maximum 5MB allowed", "Please select a file"                    |
| Error       | Search | Friendly, retryable card (timeout / can't reach server / unavailable / server error) |
| Error       | Upload | Backend error codes mapped to friendly copy with retry                               |
| Degradation | Search | Amber notice when AI re-rank/summary is temporarily unavailable                      |

---

## ## 6. End-to-End Testing

``npm run build`` passes (TypeScript + Vite) with no errors. The full pipeline was verified live against the running backend. Each feature was tested one-by-one in the UI and confirmed:

| # | Feature | How tested | Result |
|---|---------|------------|--------|
|---|---------|------------|--------|

|---|-----|-----|-----|

| 1 | Re-ranking | Query returned ranked #1–#5 with descending scores (0.950 → 0.850) + "why" reasons | ☒ |

| 2 | Deduplication | Hybrid query showed "Deduplicated — N merged" line + "Keyword + Semantic" badges, no duplicate cards | ☒ |

| 3 | Summarization | "FIT SUMMARY" boxes on cards + full summary in modal; toggling Summarize off removed them | ☒ |

| 4 | End-to-end | One search produced header + dedup line + scores + reasons + summaries together | ☒ |

| 5 | Display clarity | Clean layout + modal (open/close via ×, overlay, Esc); off-topic query returned honest low scores | ☒ |

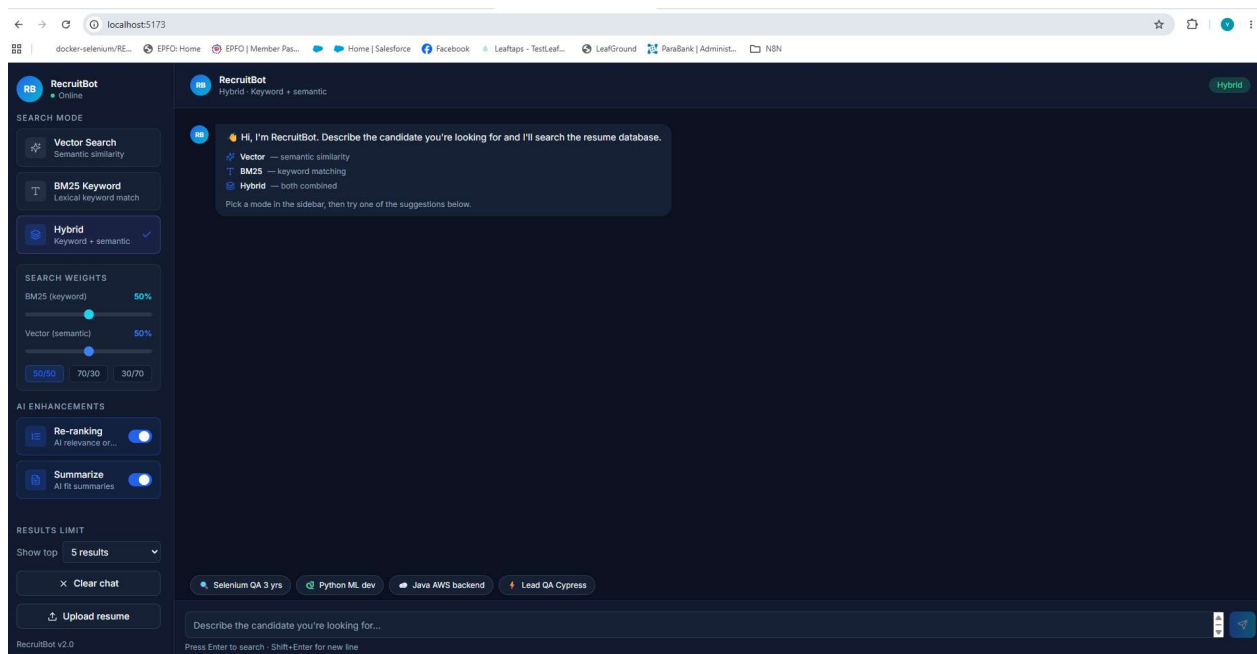
**\*\*Typical successful run timings:\*\*** embedding ~0.3–1s, BM25 ~0.15s, vector ~0.1s, re-rank ~2.5–3.7s, summarize ~0.9–2s, total ~4–6s.

---

## ## 7. Screenshots

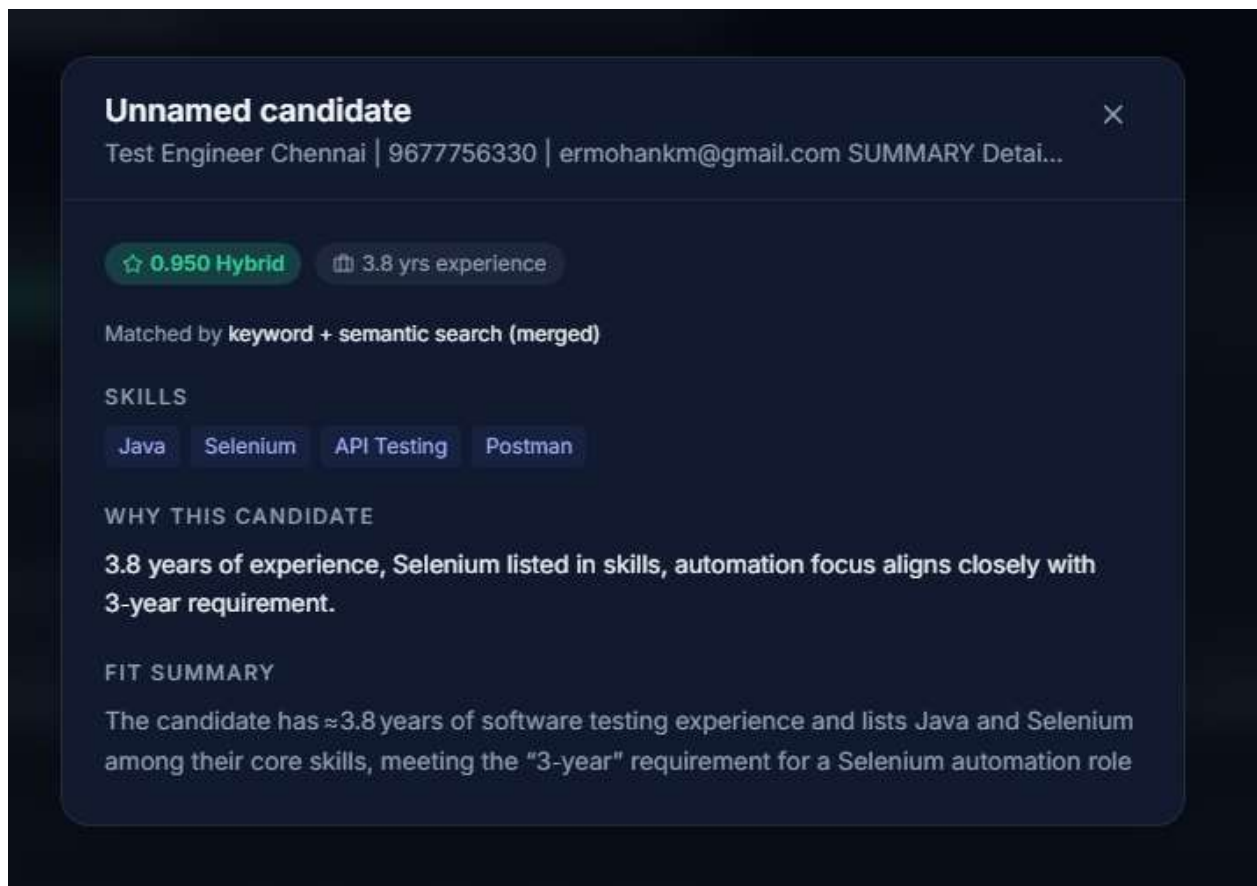
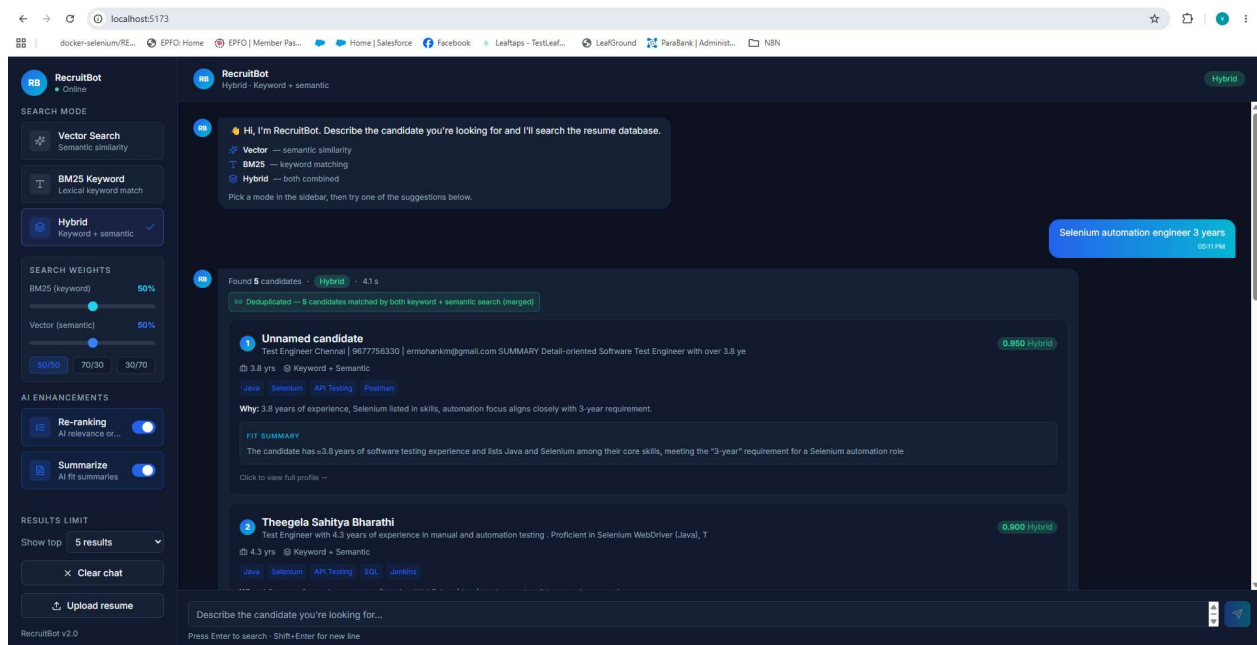
> Insert your screenshots below.

### 1. **\*\*Home / welcome screen\*\***

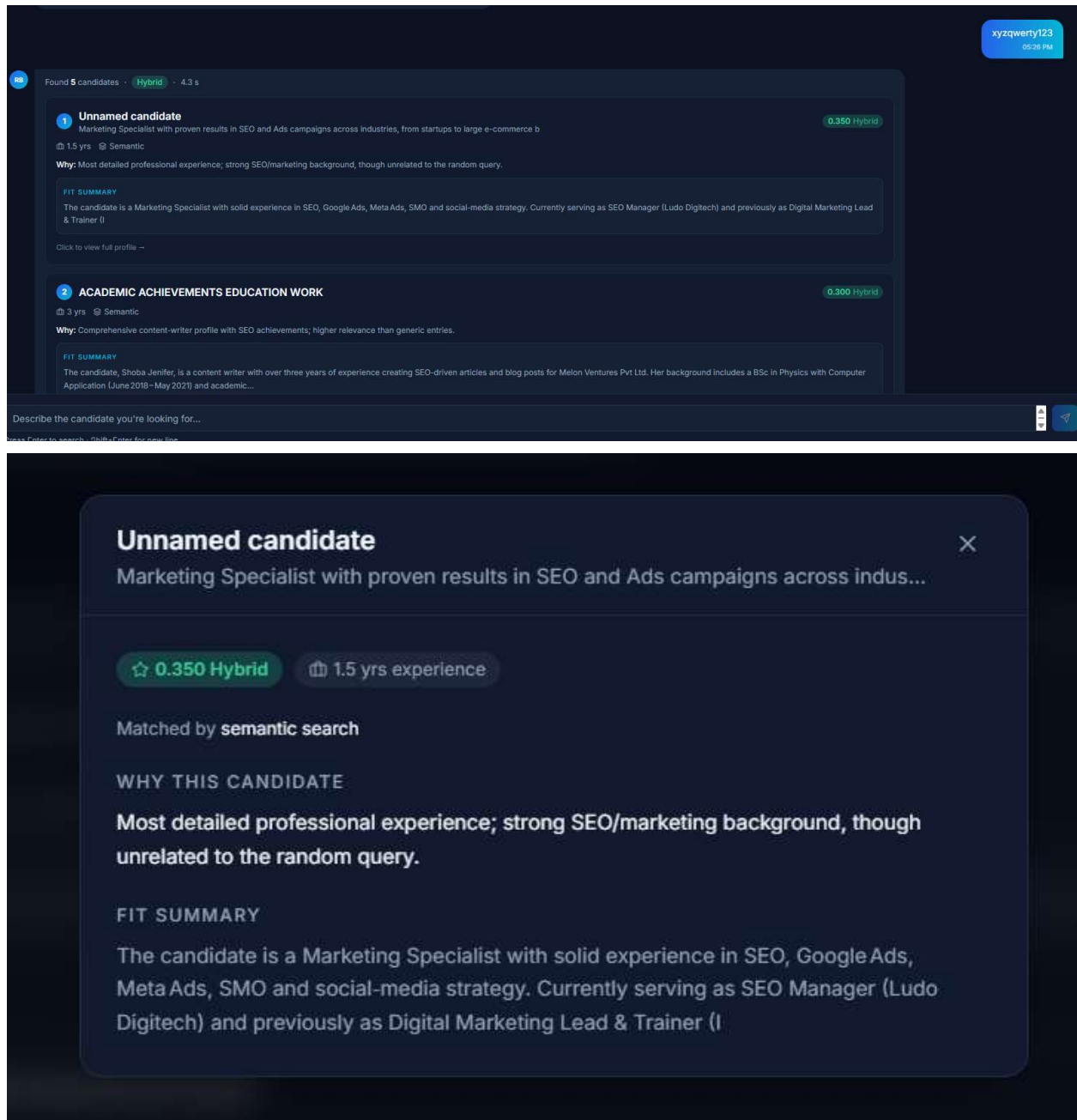


2. **\*\*Re-ranking\*\*** — ranked cards with scores + reasons, 3. **\*\*Deduplication\*\*** — "Deduplicated — N merged" line + badges, 4. **\*\*Summarization\*\*** — card with FIT SUMMARY box, 5. **\*\*Candidate modal\*\*** — score + matched-by + skills + why + full fit summary, 6. **\*\*End-to-end\*\***

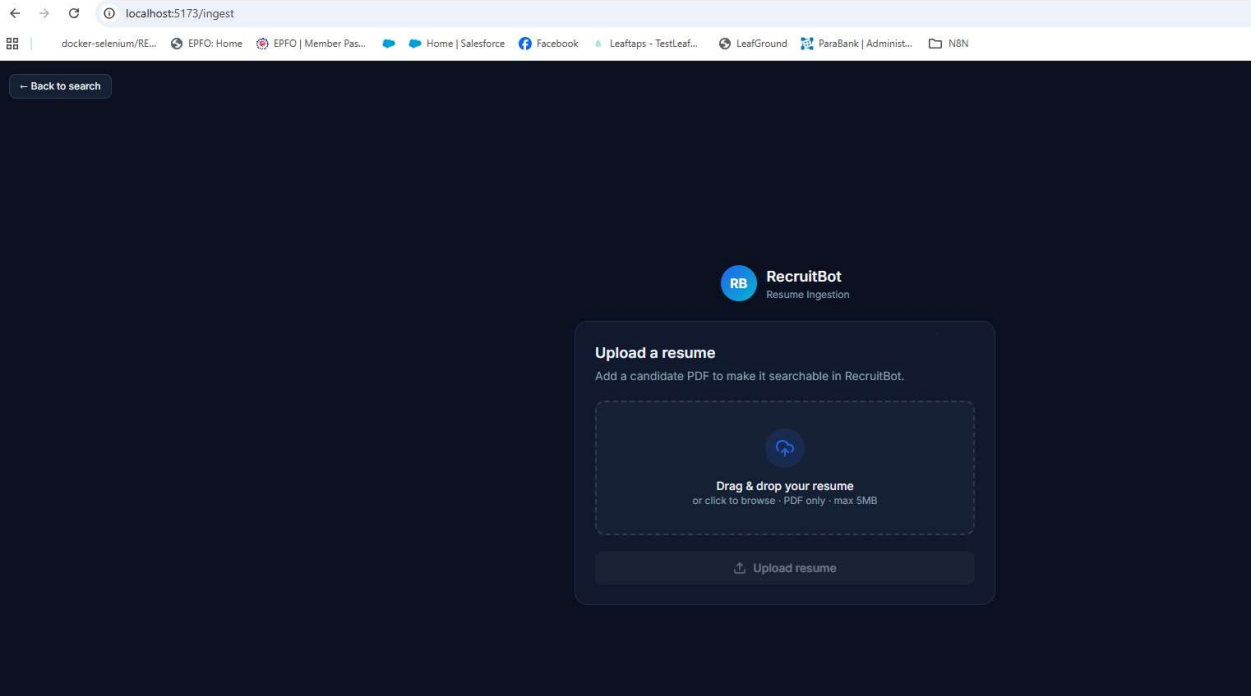
— one search showing everything together,



7. **\*\*Empty / edge case\*\*** — off-topic query with honest low scores



8. **\*\*Resume upload\*\*** (`/ingest`)



## Upload a resume

Add a candidate PDF to make it searchable in RecruitBot.



### Ingestion complete

Abish - Marketing (1) (1).pdf



Resume uploaded successfully



Embedding generated (1024-dim · mistral-embed)



MongoDB ingestion completed



Vector search ready

ROLE

Marketing Specialist with pro...

EXPERIENCE

1.5 yrs

SKILLS PARSED

0

RESUME ID

6aa149bf7d5da8768355a858

PROCESSED IN

816 ms



Upload another

9. \*\*Responsive (mobile drawer)\*\* — optional



\_\_<screenshot>\_\_

RB

RecruitBot

Resume Ingestion

Upload a resume

Add a candidate PDF to make it searchable in RecruitBot.

Ingestion complete

Abish - Marketing (1) (1).pdf

Resume uploaded successfully

Embedding generated (1024-dim · mistral-embed)

MongoDB ingestion completed

Vector search ready

ROLE

Marketing Specialist with pro...

EXPERIENCE

1.5 yrs

SKILLS PARSED

0

RESUME ID

6aa149bf7d5da8768355a858

PROCESSED IN

816 ms

Upload another

---

## ## 8. How to Run

The backend needs a `.env` with a MongoDB Atlas URI, a Mistral API key, and a Groq API key. (Never commit `.env` — it is git-ignored.)

```
``bash
# Backend → http://localhost:3000
cd resume-rag-backend
npm install
npm run dev
```

```
# Frontend → http://localhost:5173
cd recruitbot-web
npm install
npm run dev
``
```

- Candidate search: `http://localhost:5173/`
- Resume upload: `http://localhost:5173/ingest`

---

## ## 9. Repository & Submission

- **GitHub repository:** `<your repo url>`
- **Submitted to:** WhatsApp group + GitHub

Supporting documents in the repo:

- `IMPLEMENTATION.md` — phase-wise implementation detail
- `PROMPTS.md` — prompt file
- `SUBMISSION.md` — submission checklist
- `README.md` — project overview

---

## ## 10. Honest Notes

- The Groq LLM (free tier) can rate-limit intermittently. The app degrades gracefully (shows a notice) and works on retry after a short wait.
- Some candidate names appear as extracted text fragments due to backend resume parsing on messy PDFs — a data/parsing matter, not a UI issue.
- The same person uploaded twice appears as two resume ids; deduplication merges per document (same document is merged correctly).
- The candidate modal is populated from the search result (the backend does not expose a separate candidate-detail endpoint).